Failure (bad key + cost comparison) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `auth_and_cost_practice.py` : two failures worth seeing on purpose, once: a wrong API key (so you recognize the specific `openai.AuthenticationError` exception instead of a scary generic crash), and a system prompt quietly costing far more than it looks like — measured now with Doc03's cost formula driven by a real `response.usage.total_tokens` count instead of hand math. **If not:** the Build Task's "handle a bad key without a scary stack trace" requirement would ask you to catch an error type you'd never actually triggered, and a bloated system prompt in a real app could go unnoticed until the bill arrives.

Solution

```
# auth_and_cost_practice.py
from dotenv import load_dotenv
import openai
from openai import OpenAI

load_dotenv()
client = OpenAI()

def try_bad_key() -> None:
    # why: catching the specific AuthenticationError is what makes a bad
    # key fixable — a bare except would hide the real problem from you too.
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed — check your API key.")

def compare_prompt_cost(question: str) -> None:
    # why: same question, same model — the only thing that changes is the
    # system prompt's length, so any token difference is caused by that alone.
    short_prompt = "Be helpful."
    long_prompt = (
        "You are a friendly, detailed, thorough assistant "
```

```
        "who explains everything carefully. "
    ) * 10

    short_messages = [
        {"role": "system", "content": short_prompt},
        {"role": "user", "content": question},
    ]
    long_messages = [
        {"role": "system", "content": long_prompt},
        {"role": "user", "content": question},
    ]
    short = client.chat.completions.create(
        model="gpt-4o-mini", messages=short_messages
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini", messages=long_messages
    )

    # how: usage.total_tokens is the real, billed count — not an estimate.
    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt:  {long.usage.total_tokens} tokens")

try_bad_key()
compare_prompt_cost("What's the capital of France?")
```

Expected output:

```
Login failed — check your API key.
Short system prompt: 24 tokens
Long system prompt:  187 tokens
```